B. Digital Text ("Papers") Workflow

We construct a schema from a publication using only evidence in that publication. Dataset files, repository documentation and human annotations are excluded from paper-side prompts. A separate workflow extracts a schema from the released dataset (Section II-C). Both use the same schema representation and produce two outputs: a schema for comparison and a separate evidence record for inspecting its sources and decisions.

The paper workflow establishes names and table membership before adding attributes. We describe it in three stages. A *stage* contains related steps; a *step* is one minimal operation, such as reading a passage, requesting a decision or checking a response. Review can return a stage to its source evidence for bounded repair, as described in Section II-D.

1) Stage 1: Prepare passage evidence: We extract the PDF text and record the source identity, physical page numbers and character intervals. These locators allow later claims to be checked against the original input. A relevance index helps locate passages that may describe a schema. Uncertain passages are treated conservatively as relevant. The index guides navigation; name discovery still scans all source passages, including those marked irrelevant.

2) Stage 2: Discover names and group tables: The LLM proposes candidate dataset views, fields and descriptive elements from the paper. It then keeps, merges or excludes candidates using the entities represented by the records, their level of detail, and the files and relationships described in the source. A printed table may describe part of a dataset without being a dataset table itself, and one dataset view may be described in several passages. Grouping resolves these distinctions before attribute enrichment. The result is an inventory of retained names and their table membership, with supporting evidence.

3) Stage 3: Enrich attributes: For small batches of established objects, the LLM receives focused source passages and fills the requested attributes, such as datatype, unit or encoding. Each attribute requires its own supporting evidence or an explicit unknown decision. A supported name does not establish its datatype or unit. Unsupported optional attributes remain unknown rather than being completed from general knowledge or the released dataset. Because a focused request may receive only selected passages, an unknown value does not prove that the information is absent from the full paper.

C. Schema Extraction from Dataset

The dataset workflow uses the released files and their supporting documentation. Native adapters read structural facts, while an LLM interprets their meaning and enriches the schema. Schema assembly is deterministic. As on the paper side, names and structure are established before attributes are filled, and the schema and its evidence are retained separately.

1) Stage 1: Read native facts and fix the sample: Adapters record the physical column inventory, column order and available native type metadata. We preserve the original source bytes and retain a fixed sample of at most 200 rows per selected table or file using random seed 42, keeping all rows when fewer are available. Every subsequent step uses that same saved sample. Claims based on sampled values therefore describe those observations and do not establish constraints over the entire dataset.

2) Stage 2: Establish names and bind documentation: Physical columns determine the field inventory and order. Position-based identifiers distinguish columns with duplicate names. Supporting documentation is associated with the corresponding tables and fields, so semantic decisions can refer to both native facts and relevant explanations. The resulting inventory fixes the objects available for attribute filling.

3) Stage 3: Enrich attributes: The LLM uses the native readout, fixed sample and associated documentation to interpret fields and fill the configured attributes. Evidence records distinguish explicit declarations, direct observations and inferences. For example, an observed string value is different from a declared type constraint, and a unit inferred from a name is different from an explicitly documented unit. Unsupported optional attributes remain unknown. The shared harness reviews these decisions and assembles accepted outputs.

D. Workflow Harness Details

The harness coordinates source access, model requests, review and export for both workflows. It preserves the evidence behind names, table membership and attributes so that the final schema can be inspected at each level.

1) Contract-bound requests and review: Model requests use a fixed JSON output contract specifying the required decisions and evidence. Attribute responses must contain the requested decision keys and refer only to objects in the established inventory. Missing keys and newly introduced names are rejected. Accepted claims retain a value, provenance status, reason and source references. Provenance distinguishes `declared`, `observed` and `inferred` information; unresolved decisions are exported as `unknown`.

A separate review checks proposed decisions against their evidence and returns feedback for bounded repair. Final checks cover names, structure and claims; failed checks block acceptance. Unresolved optional attributes and deferred source references remain visible. The harness retains source views and model responses, allowing a reader to inspect both the selected values and the decisions behind them. Contract and citation checks establish that an output is well formed and source-linked; they do not guarantee that its meaning is correct.

2) Schema and evidence outputs: Accepted decisions are assembled deterministically into `full-schema.json`. The schema describes datasets, tables and fields, with structure, value and encoding attributes. This is the representation used for subsequent paper–dataset comparison. The separate `evidence.json` retains supporting evidence, conflicts, coverage and unresolved decisions. Keeping both outputs makes the result useful as a schema and as an inspectable record of how it was obtained.

The compact example below selects saved records from an accepted MIT Supercloud dataset extraction. The outer grouping keys are for presentation; `support_excerpt` abbreviates the saved name quotation. Other shown keys and values are retained from the records.

```
{
  "schema_field": {
    "name": "Node",
    "value": {"datatype": "string"}
  },
  "retention": {
    "field_handle": "f0001",
    "table_handle": "table", "native_keep": true
  },
  "evidence": {"Node": {
    "name": {
      "status": "declared",
      "location": {
        "section": "field-0001.json#page-1",
        "physical_page": 1,
        "start_char": 0, "end_char": 1500
      },
      "support_excerpt": "\"name\":\"Node\""
    },
    "value.datatype": {
      "status": "observed",
      "location": {
        "section": "field-0001.json#row-1",
        "physical_page": 1,
        "start_char": 285, "end_char": 303
      },
      "support": "\"r2652301-n830961\""
    }
  }}
}
```

The field's retained table membership, declared name and observed datatype have separate records. Here page 1 is a page of the generated dataset readout. The string quotation supports the sample-based datatype decision; it does not establish a population-wide constraint.